Practical 5

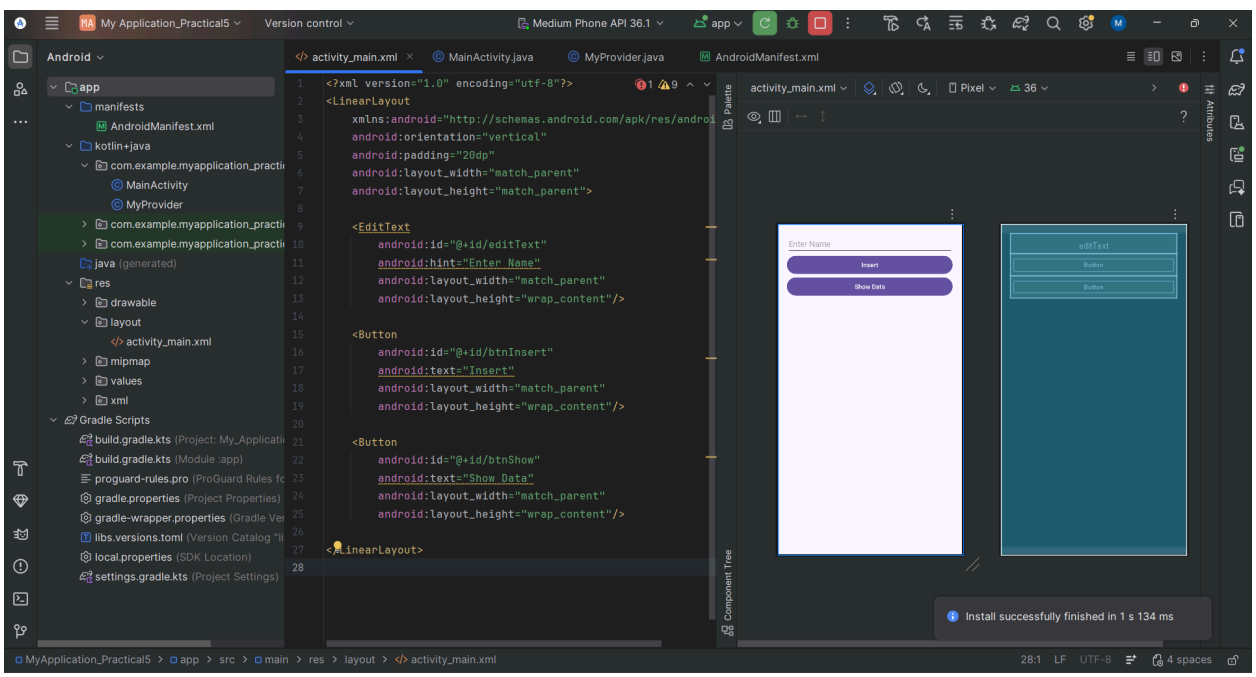
Aim: Create a content provider to manage and share data between different components of your Android application.

Theory: This practical demonstrates the use of a Content Provider to manage and share data between application components. It shows how data can be inserted and retrieved using ContentResolver and URIs. It also explains operations like insert and query using in-memory storage.

Codes:

activity_main.xml - Provides UI components:

- EditText → Enter name
- Insert Button → Add data
- Show Data Button → Display stored data



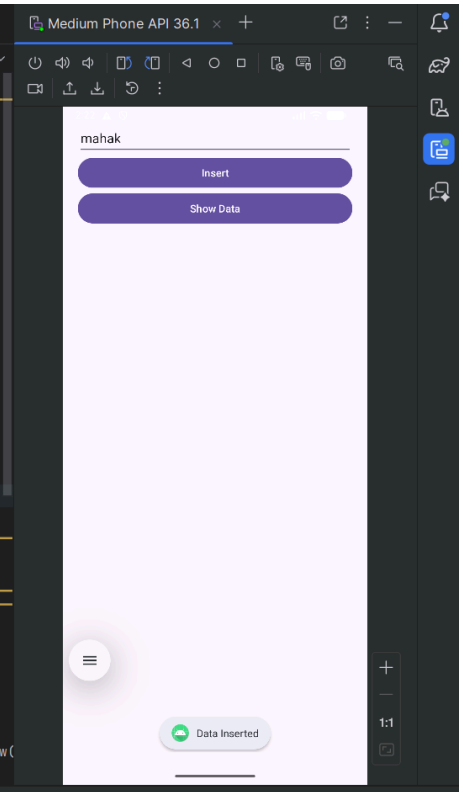
AndroidManifest.xml - Registers the Content Provider with a unique authority (com.example.provider) so it can be accessed by the app.

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication_Practical5">

        <!-- Content Provider -->
        <provider
            android:name=".MyProvider"
            android:authorities="com.example.provider"
            android:exported="true"/>

        <!-- Main Activity -->
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

Output:

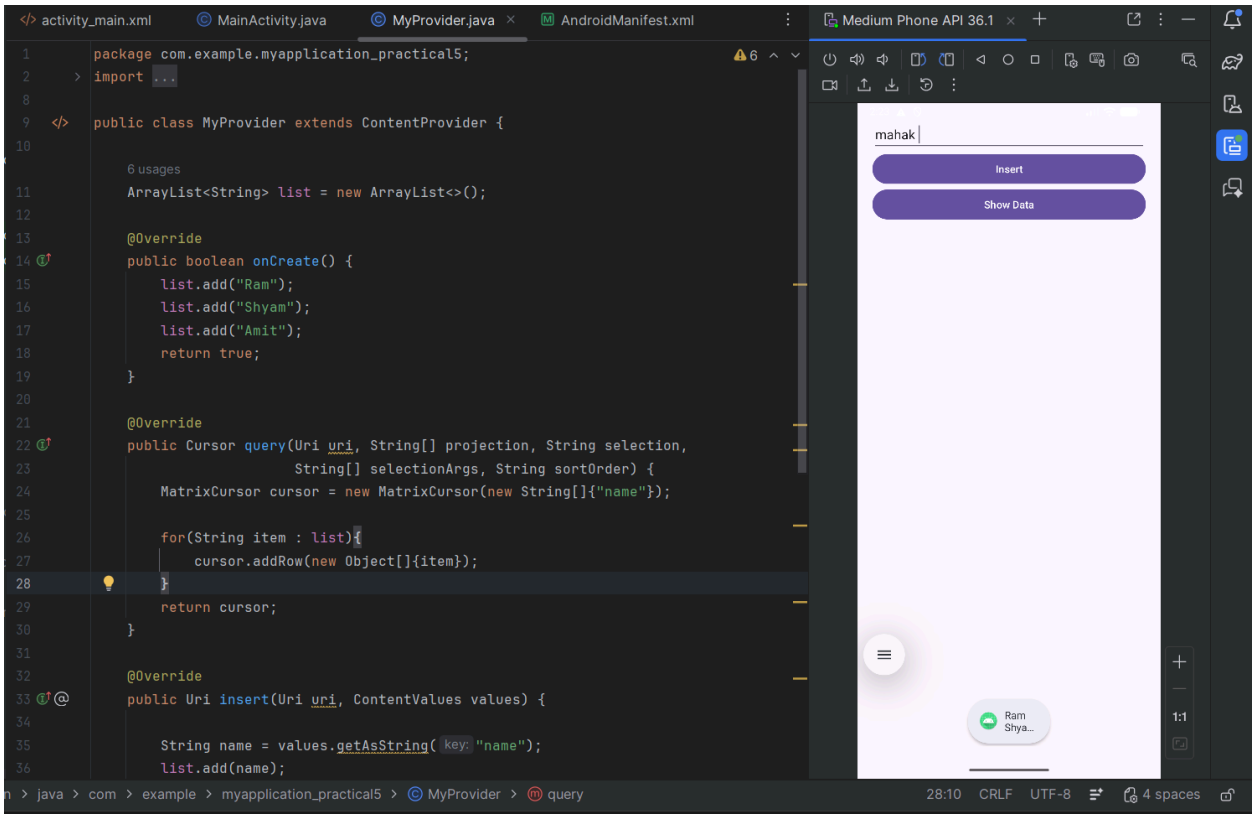
MainActivity.java - This activity interacts with the Content Provider using ContentResolver. It allows inserting and retrieving data from the provider.

```java
package com.example.myapplication_practical5;

import ...

public class MainActivity extends AppCompatActivity {

    2 usages
    EditText editText;
    2 usages
    Button btnInsert, btnShow;

    2 usages
    Uri uri = Uri.parse( uriString: "content://com.example.provider");

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        editText = findViewById(R.id.editText);
        btnInsert = findViewById(R.id.btnInsert);
        btnShow = findViewById(R.id.btnShow);

        btnInsert.setOnClickListener( View v -> {

            ContentValues values = new ContentValues();
            values.put("name", editText.getText().toString());

            getContentResolver().insert(uri, values);

            Toast.makeText( context: this, text: "Data Inserted",Toast.LENGTH_SHORT).show(
        });
```



Code Explanation (MainActivity.java)

- ContentValues → Used to store data before insertion
- insert() → Adds data to Content Provider
- query() → Retrieves data
- Cursor → Iterates through data
- Toast → Displays results



MyProvider.java - This class implements a custom Content Provider that stores and manages data using an ArrayList. It allows insertion, retrieval, and deletion of data through standard ContentProvider methods.

Code Explanation (MyProvider.java)

- onCreate() → Initialises data (adds default names)
- ArrayList → Stores data in memory
- query() → Returns all stored data using MatrixCursor
- insert() → Adds new data to the list
- delete() → Clears all data
- update() → Not implemented
- getType() → Returns null

Conclusion

The Content Provider was successfully implemented to manage and share data. Data insertion and retrieval were performed using ContentResolver, demonstrating how components communicate using a standard interface.